

Lec 11

4th Aug, 2026

2D Arrays in DSA — Complete Notes: Theory, Address Calculation, Solved Problems & C++ Implementation

1. What Is a 2D Array, Really?

A "2D array" is a **conceptual** abstraction. Physical RAM is a single linear sequence of byte addresses — there is no such thing as a literal grid in memory. When you write `int arr[3][4]`, the compiler:

1. Reserves `ROWS * COLS * sizeof(int)` contiguous bytes.
2. Silently translates every `arr[i][j]` access into a 1D offset computation.
3. Never actually "walks" a grid — every access is $O(1)$ pointer arithmetic, not a search.

Understanding this translation is the entire point of the address-calculation topic: it proves *why* 2D array access is constant-time and *how* the compiler does it under the hood.

1.1 Two physical layout strategies

Layout	Storage order	Used by
Row-major	Row 0 fully, then row 1 fully, then row 2...	C, C++, Java, Python (lists of lists conceptually), Pascal-descendants excluded
Column-major	Column 0 fully, then column 1 fully...	Fortran, MATLAB, R, older linear-algebra libraries

C++ **always** uses row-major order for built-in arrays. Column-major appears in these notes only for contrast and exam completeness.

1.2 Why row-major matters for performance (cache locality)

Because row-major storage places `arr[i][j]` and `arr[i][j+1]` adjacent in memory, looping with `j` as the inner loop variable reads memory sequentially — this is **cache-friendly**. Looping with `i` as the inner variable jumps `COLS * w` bytes on every step — this causes more cache misses. This is a common interview/viva follow-up question tied directly to the address formula.

```
// Cache-friendly (row-major traversal matches storage order)
for (int i = 0; i < ROWS; ++i)
    for (int j = 0; j < COLS; ++j)
        sum += arr[i][j];

// Cache-unfriendly (jumps COLS*W bytes each inner step)
for (int j = 0; j < COLS; ++j)
    for (int i = 0; i < ROWS; ++i)
        sum += arr[i][j];
```

1.3 Static 2D arrays specifically

A **static 2D array** (`int arr[ROWS][COLS]`) is allocated as **one single contiguous block** at compile time — no separate allocations, no fragmentation, no pointer indirection. This is precisely why the address formula in Section 2 applies to it directly and exactly: the compiler generates `arr[i][j]` accesses using that exact arithmetic internally. This is also why static arrays are the cleanest vehicle for teaching/proving the address formula — there is no ambiguity about contiguity the way there can be with heap-allocated jagged (`int**`) structures. All C++ code in these notes (Section 5) works with static arrays only, for this reason.

2. Address Calculation — The Algorithm

Goal: Find the memory address of element `arr[i][j]` in a 2D array of size `ROWS x COLS`, given the base address and element size, WITHOUT dereferencing the array.

2.1 Step-by-step algorithm (Row-Major Order)

INPUT : B (base address), ROWS, COLS, W (size of one element in bytes), i, j
 OUTPUT: Address of `arr[i][j]`

Step 1: Validate indices $\rightarrow 0 \leq i < \text{ROWS}$ and $0 \leq j < \text{COLS}$

Step 2: Number of FULL rows to skip completely = i

Step 3: Each full row occupies (`COLS * W`) bytes

$\rightarrow$ bytes to skip for rows = $i * \text{COLS} * W$

Step 4: Within the target row, skip j elements

$\rightarrow$ bytes to skip within row = $j * W$

Step 5: Total offset (in bytes) = $(i * \text{COLS} * W) + (j * W)$
 $= (i * \text{COLS} + j) * W$

Step 6: Address = B + Total offset

Step 7: Return Address

Complexity: $O(1)$ time, $O(1)$ space — no loops, no traversal, just arithmetic. This is the core insight examiners want you to state explicitly: 2D array indexing is constant time regardless of ROWS or COLS.

2.2 The Formula

```
Row-major:    Address(i, j) = B + [ (i * COLS) + j ] * W
Column-major: Address(i, j) = B + [ (j * ROWS) + i ] * W
```

Where:

- B = address of `arr[0][0]` (base address)
- i = row index (0-based), j = column index (0-based)
- $ROWS$, $COLS$ = dimensions
- W = `sizeof(element type)`

2.3 Why it works (intuition)

Think of the 2D array as a book: `ROWS` chapters (rows), each with `COLS` words (elements). To find word j of chapter i , you don't need to know how many *total* chapters the book has — you only need to know how long **one chapter** is (`COLS`), skip i of those, then walk j more words into the current chapter. This is exactly why the formula uses `COLS` (not `ROWS`) as the multiplier — the row-major formula is independent of `ROWS` entirely, which surprises many students on first encounter and is worth remembering as a standalone fact.

2.4 Generalization to 3D and n-D arrays (bonus, often asked as an extension)

For a 3D array `arr[D1][D2][D3]` in row-major (C-style) order:

```
Address(i, j, k) = B + [ (i * D2 * D3) + (j * D3) + k ] * W
```

Pattern: each index's multiplier is the product of the sizes of *all dimensions to its right*. This generalizes to any n-D array and is a natural follow-up question once the 2D case is understood.

3. Fully Solved Practice Questions

Q1 — Basic row-major lookup

`int arr[5][6]` , base address `B = 2000` , `sizeof(int) = 4` . Find address of `arr[2][3]` .

Solution (fully worked):

```
ROWS = 5, COLS = 6, W = 4, i = 2, j = 3
Step 1: Validate -> 0<=2<5 OK, 0<=3<6 OK
Step 2: rows to skip = i = 2
Step 3: bytes from full rows = i * COLS * W = 2 * 6 * 4 = 48
Step 4: bytes within target row = j * W = 3 * 4 = 12
Step 5: total offset = 48 + 12 = 60
Step 6: Address = 2000 + 60 = 2060
```

Answer: 2060

Q2 — Different element type

`float mat[10][8]` , base address `5000` , `sizeof(float) = 4` . Find address of `mat[4][7]` .

Solution:

```
i=4, j=7, COLS=8, W=4
offset = (4*8 + 7) * 4 = (32+7)*4 = 39*4 = 156
Address = 5000 + 156 = 5156
```

Answer: 5156

Q3 — Column-major (contrast case)

`char grid[100][50]` , base address `1000` , **column-major**, `sizeof(char) = 1` . Find address of `grid[3][10]` .

Solution:

```
Column-major formula: Address = B + (j*ROWS + i) * W
ROWS = 100, i = 3, j = 10, W = 1
offset = (10*100 + 3) * 1 = 1003
Address = 1000 + 1003 = 2003
```

Answer: 2003 (Note: if this were computed as row-major by mistake, you'd get $1000 + (3_50+10)_1 = 1160$ — a common exam trap. Always check which order the question specifies.)

Q4 — Verification / "spot the error" drill

Given `Address(2,2) = 1348` claimed for a row-major `int arr[R][6]` with base `1300`, `sizeof(int)=4`. Verify.

Solution:

```
Expected = 1300 + (2*6 + 2) * 4 = 1300 + 14*4 = 1300 + 56 = 1356
Claimed   = 1348
1356 ≠ 1348 -> claim is INCORRECT
```

Answer: Mismatch — the correct address is 1356, not 1348.

Q5 — Row-boundary adjacency

`int arr[4][5]`. How many bytes are skipped moving from `arr[1][4]` to `arr[2][0]` (the very next element in row-major order)?

Solution:

```
Address(1,4) = B + (1*5+4)*4 = B + 36
Address(2,0) = B + (2*5+0)*4 = B + 40
Difference = 40 - 36 = 4 bytes = 1 * sizeof(int)
```

Answer: 4 bytes. This proves the last element of row `i` and the first element of row `i+1` are physically adjacent — row-major storage has no "gap" between rows.

Q6 — Full address table + stride verification

`int arr[3][3]`, base `4000`, `sizeof(int)=4`. List every address and confirm constant stride.

Solution:

```
(0,0): 4000 + (0*3+0)*4 = 4000
(0,1): 4000 + (0*3+1)*4 = 4004
(0,2): 4000 + (0*3+2)*4 = 4008
(1,0): 4000 + (1*3+0)*4 = 4012
(1,1): 4000 + (1*3+1)*4 = 4016
(1,2): 4000 + (1*3+2)*4 = 4020
(2,0): 4000 + (2*3+0)*4 = 4024
(2,1): 4000 + (2*3+1)*4 = 4028
(2,2): 4000 + (2*3+2)*4 = 4032
```

Answer: Constant stride of 4 bytes between every consecutive element — confirms full contiguity.

Q7 — Reverse problem: given an address, find (i, j) (*new — commonly asked variant not in original notes*)

`int arr[6][7]`, base `3000`, `sizeof(int)=4`. An element is stored at address `3080`. Find `(i, j)`.

Solution:

```
offset_bytes = 3080 - 3000 = 80
offset_elements = 80 / 4 = 20
i = offset_elements / COLS = 20 / 7 = 2 (integer division)
j = offset_elements % COLS = 20 % 7 = 6
Verify: (2*7+6)*4 + 3000 = (14+6)*4+3000 = 80+3000 = 3080 ✓
```

Answer: `arr[2][6]`

Q8 — Reverse problem, column-major (*new*)

`double m[20][15]`, base `10000`, column-major, `sizeof(double)=8`. Address `10000 + 3200` was accessed. Find `(i, j)`.

Solution:

```
offset_bytes = 3200, offset_elements = 3200/8 = 400
Column-major: offset = j*ROWS + i -> 400 = j*20 + i
j = 400 / 20 = 20 -> but valid j range is 0..14, so re-check:
Actually 400 / 20 = 20 exactly, i = 0
j must be < 15, so j=20 is OUT OF RANGE -> the given offset is invalid for a
20x15 matrix.
```

Answer: Invalid offset — exceeds array bounds (deliberately included as a bounds-checking drill; always verify `j < COLS` / `i < ROWS` after solving).

Q9 — Sub-block / partial row start address (*new — practical variant*)

`int arr[8][10]`, base `500`, `sizeof(int)=4`. Find the starting address of **row 5** (i.e., `arr[5][0]`), without computing column offset separately.

Solution:

```
Row-start shortcut: Address(row_start) = B + i * COLS * W
= 500 + 5 * 10 * 4 = 500 + 200 = 700
```

Answer: 700. This shortcut (`j` term drops to 0) is useful for row-pointer arithmetic, e.g. `int* rowPtr = (int*)((char*)arr + i*COLS*sizeof(int));`

4. Common Pitfalls (exam & viva focus)

1. **Confusing ROWS and COLS in the formula.** The row-major multiplier is always `COLS` , never `ROWS` — a very common transcription error under time pressure.
 2. **Forgetting to multiply by `w`.** The bracketed term `(i*COLS+j)` is an **element count**, not a byte count — it must be multiplied by `sizeof(type)` to become an address offset.
 3. **Off-by-one in 0-based vs 1-based indexing.** All formulas here assume `i, j` are 0-based (C++/C convention). If a question uses 1-based indexing, subtract 1 from each index before applying the formula.
-

5. Complete C++ Implementation (Static Arrays Only)

Pure static 2D array implementation — no `new / delete` , no heap allocation, no `int**` .
Improvements over a bare-minimum version: bounds checking with exceptions, `const` - correctness, and an explicit reverse-lookup function (`address → (i,j)`) that proves the formula is algebraically invertible — all organized in one namespace with clean struct-based design (no templates, no `std::vector` , per spec).

```
#include <iostream>
#include <stdexcept>
#include <cstdint>

// =====
// namespace dsa2d - all 2D array / address-calculation logic
// =====
namespace dsa2d
{
    constexpr int ROWS = 3;
    constexpr int COLS = 4;

    // -----
    // Struct 1: Static 2D array (single contiguous block)
    // -----
    struct StaticMatrix
    {
        int data[ROWS][COLS]{};
    };
}
```

```

void fillSequential()
{
    int val = 0;
    for (int i = 0; i < ROWS; ++i)
        for (int j = 0; j < COLS; ++j)
            data[i][j] = val++;
}

void print() const
{
    for (int i = 0; i < ROWS; ++i)
    {
        for (int j = 0; j < COLS; ++j)
            std::cout << data[i][j] << "\t";
        std::cout << "\n";
    }
}

void validate(int i, int j) const
{
    if (i < 0 || i >= ROWS || j < 0 || j >= COLS)
        throw std::out_of_range("StaticMatrix: index out of range");
}

// Address as given directly by the compiler
const int* addressCompiler(int i, int j) const
{
    validate(i, j);
    return &data[i][j];
}

// Address using the manual row-major formula:
// B + (i * COLS + j) * sizeof(int)
const int* addressFormula(int i, int j) const
{
    validate(i, j);
    const char* base = reinterpret_cast<const char*>(data);
    std::size_t offset = (static_cast<std::size_t>(i) * COLS + j) *
sizeof(int);
    return reinterpret_cast<const int*>(base + offset);
}

// Reverse lookup: given a valid element pointer, recover (i, j)
void reverseLookup(const int* ptr, int& i, int& j) const
{
    const char* base = reinterpret_cast<const char*>(data);

```

```

        std::ptrdiff_t byteOffset = reinterpret_cast<const char*>(ptr) -
base;

        if (byteOffset < 0 || byteOffset % sizeof(int) != 0)
            throw std::invalid_argument("reverseLookup: pointer misaligned
or out of range");
        std::size_t elemOffset = static_cast<std::size_t>(byteOffset) /
sizeof(int);
        i = static_cast<int>(elemOffset / COLS);
        j = static_cast<int>(elemOffset % COLS);
        validate(i, j);
    }

    // Row-start shortcut address (Q9-style): j term = 0
    const int* rowStartAddress(int i) const
    {
        return addressFormula(i, 0);
    }
};

// -----
// Utility: prints a label and a raw memory address
// -----
void printAddress(const char* label, const void* ptr)
{
    std::cout << label << " = " << ptr << "\n";
}

} // namespace dsa2d

// =====
// Driver code
// =====
int main()
{
    using namespace dsa2d;

    std::cout << "=== 1. Static 2D Array ===\n";
    StaticMatrix sm;
    sm.fillSequential();
    sm.print();

    for (int i = 0; i < ROWS; ++i)
    {
        for (int j = 0; j < COLS; ++j)
        {
            const int* compilerAddr = sm.addressCompiler(i, j);

```

```

        const int* formulaAddr = sm.addressFormula(i, j);
        std::cout << "(" << i << "," << j << ") compiler=" << compilerAddr
                    << " formula=" << formulaAddr
                    << " match=" << (compilerAddr == formulaAddr ? "true" :
"false") << "\n";
    }
}

// Reverse lookup demo
int ri, rj;
sm.reverseLookup(sm.addressFormula(2, 1), ri, rj);
std::cout << "Reverse lookup of arr[2][1]'s address -> (" << ri << "," <<
rj << ")\n";

std::cout << "\n== 2. Row-start shortcut addresses ==\n";
for (int i = 0; i < ROWS; ++i)
    printAddress("Row start", sm.rowStartAddress(i));

std::cout << "\n== 3. Stride verification (constant sizeof(int) gap)
==\n";
for (int i = 0; i < ROWS; ++i)
    for (int j = 0; j < COLS; ++j)
        printAddress("Element address", sm.addressFormula(i, j));

std::cout << "\n== 4. Bounds-checking demo ==\n";
try
{
    sm.addressFormula(ROWS, 0); // deliberately out of range
}
catch (const std::out_of_range& e)
{
    std::cout << "Caught expected exception: " << e.what() << "\n";
}

return 0;
}

```

Compile & run:

```

g++ -std=c++17 -O2 -Wall -Wextra 2d_array_dsa.cpp -o 2d_array_dsa
./2d_array_dsa

```

5.1 What changed vs. a minimal version, and why

Feature	Why it matters
<code>validate() + std::out_of_range</code>	Turns silent undefined behavior (out-of-bounds access) into a catchable, debuggable exception — mirrors how <code>std::array::at()</code> / <code>std::vector::at()</code> behave.
<code>reverseLookup()</code>	Directly demonstrates the algebraic inverse of the address formula (Q7/Q8 above) in working code, not just on paper.
<code>rowStartAddress()</code>	Implements the Q9 shortcut (<code>j</code> term = 0) as reusable, tested code instead of a one-off manual calculation.
<code>const</code> -correctness	Read-only operations (<code>print</code> , <code>addressFormula</code> , <code>addressCompiler</code>) are marked <code>const</code> , documenting intent and preventing accidental mutation through those calls.
Zero heap allocation	No <code>new</code> / <code>delete</code> , no <code>int**</code> , no manual memory management — the entire struct lives on the stack, which is the whole point of a "static arrays only" implementation: no leak/double-free risk exists in the first place.

6. What the Output Proves

- `addressCompiler` and `addressFormula` always match — confirming the row-major formula is *exactly* what the compiler does internally for `arr[i][j]` .
- `reverseLookup` recovers `(i,j)` from a raw address, proving the formula is algebraically invertible (Section 3, Q7).
- The row-start addresses printed in `main()` increase by exactly `COLS * sizeof(int)` bytes each time (Section 3, Q9), and consecutive element addresses within a row increase by exactly `sizeof(int)` bytes (Section 3, Q6) — together these confirm the array is one single, fully contiguous block with no internal gaps.
- The bounds-checking demo confirms out-of-range `(i,j)` throws rather than silently reading/writing invalid memory.

7. Quick Reference Sheet

Row-Major Address(*i,j*) = *B* + (*i* * COLS + *j*) * *W*
 Column-Major Address(*i,j*) = *B* + (*j* * ROWS + *i*) * *W*

Reverse (row-major): *offset* = (Address - *B*) / *W*
 i = *offset* / COLS (integer division)

```
j = offset % COLS
```

Row-start shortcut: $\text{Address}(i, 0) = B + i * \text{COLS} * W$

B = base address (address of first element, `arr[0][0]`)

COLS = number of columns

ROWS = number of rows

W = `sizeof(element)`

i, j = row index, column index (0-based)

Note: all formulas and code in these notes assume a static, stack-allocated 2D array (`int arr[ROWS][COLS]`) — a single contiguous block for which the row-major formula holds exactly, everywhere.